
pAstroCORE

Release 0.0.91

Alexey Rudnitskiy

Jul 29, 2025

MODULES:

1	pastrocore package	3
1.1	Subpackages	3
1.1.1	pastrocore.base package	3
1.1.1.1	Submodules	3
1.1.1.2	Module contents	22
1.1.2	pastrocore.super package	22
1.1.2.1	Submodules	22
1.1.2.2	Module contents	28
1.1.3	pastrocore.utils package	28
1.1.3.1	Submodules	28
1.1.3.2	Module contents	30
1.2	Module contents	30
2	common package	31
2.1	Subpackages	31
2.1.1	common.base package	31
2.1.1.1	Submodules	31
2.1.1.2	Module contents	39
2.1.2	common.super package	39
2.1.2.1	Submodules	39
2.1.2.2	Module contents	45
2.1.3	common.utils package	45
2.1.3.1	Submodules	45
2.1.3.2	Module contents	49
2.2	Module contents	49
	Python Module Index	51
	Index	53

Add your content using `reStructuredText` syntax. See the [reStructuredText](#) documentation for details.

PASTROCORE PACKAGE

1.1 Subpackages

1.1.1 pastrocore.base package

1.1.1.1 Submodules

pastrocore.base.frequencies module

```
class pastrocore.base.frequencies.Frequencies (* (Keyword-only parameters separator (PEP
3102)), name: str | None = None, items:
Dict[str, IF] = None, isactive: bool = True,
use_cache: bool = False)
```

Bases: `BaseContainer[IF]`

Base class representing a collection of Intermediate Frequency (IF) objects.

Manages a dictionary of IF objects indexed by their names, ensuring no frequency range overlaps. Inherits from BaseContainer for universal collection management and serialization.

_items

Dictionary of IF objects with names as keys.

Type

`Dict[str, IF]`

name

Identifier for the Frequencies object.

Type

str, optional

isactive

Indicates whether the Frequencies object is active.

Type

bool

Notes

- Frequency ranges [freq, freq + bandwidth] must not overlap.
- IF objects must have unique non-None names.

`activate_item(name: str) → None`

Activate a specific frequency by its name.

Triggers synchronization with a parent Observation if present.

Parameters

`name (str)` – The name of the frequency to activate.

Raises

`KeyError` – If the name is not found in the collection.

`add(if_obj: IF) → None`

Add an IF object to the collection.

Parameters

`if_obj (IF)` – The IF object to add.

Raises

`ValueError` – If frequency range overlaps with existing IFs.

`copy()` → *Frequencies*

Create a deep copy of the Frequencies object.

`create_if(name: str = None, frequency: float = 1000.0, bandwidth: float = 16.0, polarizations: str | List[str] | None = None, isactive: bool = True) → None`

Create and add a new IF object to the collection.

Parameters

- `name (str, optional)` – Unique identifier for the IF. If None, a UUID-based name is generated.
- `frequency (float)` – The IF frequency in MHz. Default is 1000.0 MHz.
- `bandwidth (float)` – The bandwidth in MHz. Default is 16.0 MHz.
- `polarizations (Optional[Union[str, List[str]]])` – Polarization codes. Default is None (empty list).
- `isactive (bool)` – Whether the IF is active. Default is True.

Raises

- `ValueError` – If frequency or bandwidth is not positive, or if frequency range overlaps with existing IFs.
- `ValueError` – If polarizations are invalid or mix groups.
- `TypeError` – If polarizations contain non-string elements.

`deactivate_item(name: str) → None`

Deactivate a specific frequency by its name.

Triggers synchronization with a parent Observation if present.

Parameters

`name (str)` – The name of the frequency to deactivate.

Raises

`KeyError` – If the name is not found in the collection.

`get_bandwidths()` → List[float]

Retrieve a list of all IF bandwidths.

Returns

List of bandwidths in MHz.

Return type

List[float]

`get_frequencies() → List[float]`

Retrieve a list of all IF frequencies.

Returns

List of frequencies in MHz.

Return type

List[float]

`get_polarizations() → List[List[str]]`

Retrieve a list of all IF polarizations.

Returns

List of polarization lists.

Return type

List[List[str]]

`get_wavelengths() → List[float]`

Retrieve a list of wavelengths for all IF frequencies.

Returns

List of wavelengths in centimeters.

Return type

List[float]

`isactive: bool`

`name: str`

`set_if(name: str, frequency: float | None = None, bandwidth: float | None = None, polarizations: str | List[str] | None = None, isactive: bool | None = None) → None`

Update an existing IF object in the collection with new parameters.

Parameters

- **name** (*str*) – The name of the IF to update.
- **frequency** (*float, optional*) – The new IF frequency in MHz.
- **bandwidth** (*float, optional*) – The new bandwidth in MHz.
- **polarizations** (*Optional[Union[str, List[str]]], optional*) – The new polarization codes.
- **isactive** (*bool, optional*) – The new active status.

Raises

- **KeyError** – If the IF with the given name does not exist.
- **ValueError** – If the new frequency range overlaps with existing IFs, or if frequency/bandwidth is not positive.
- **ValueError** – If polarizations are invalid or mix groups.
- **TypeError** – If polarizations contain non-string elements.

`set_items(items: Dict[str, IF]) → None`

Set or replace all IF objects in the collection.

Parameters

items – Dictionary of IF objects with names as keys.

Raises

- **ValueError** – If any frequency range overlaps with another.

```
class pastrocore.base.frequencies.IF(*, name: str = None, frequency: float = 1000.0,
                                     bandwidth: float = 16.0, polarizations: str | List[str] | None
                                     = None, isactive: bool = True)
```

Bases: *BaseEntity*

Base class representing an Intermediate Frequency (IF) with frequency, bandwidth, and polarization properties.

name

Unique identifier for the IF.

Type

str, optional

frequency

The IF frequency in MHz. Must be positive.

Type

float

bandwidth

The bandwidth in MHz. Must be positive.

Type

float

polarizations

List of polarization codes from VALID_POLARIZATIONS.

Type

List[str]

isactive

Indicates whether the IF is active.

Type

bool

Notes

- Polarizations must belong to a single group: circular, paired linear, or single linear.
- Wavelength is calculated as C_MHZ_CM / frequency.

bandwidth: float

copy() → *IF*

Create a deep copy of the IF object.

frequency: float

get_frequency_wavelength() → float

Calculate the wavelength corresponding to the IF frequency.

Returns

Wavelength in centimeters.

Return type

float

Raises

ValueError – If frequency is zero.

`isactive: bool`

`name: str`

`polarizations: List[str]`

`set_frequency_wavelength(wavelength_cm: float) → None`
Set frequency based on a wavelength.

Parameters

`wavelength_cm(float)` – Wavelength in centimeters. Must be positive.

Raises

`ValueError` – If `wavelength_cm` is not positive.

pastrocore.base.observation module

```
class pastrocore.base.observation.Observation(name: str = None, code: str =
    'OBS_DEFAULT', sources: Sources = None,
    telescopes: Telescopes = None, frequencies:
    Frequencies = None, scans: Scans = None,
    observation_type: str = 'VLBI',
    calculated_data: Dict[str, Any] = None,
    isactive: bool = True)
```

Bases: `BaseEntity`

Base class representing an astronomical observation with sources, telescopes, frequencies, and scans.

Encapsulates the structure and metadata of an observation, such as its unique code, type (VLBI or SINGLE_DISH), and associated entities. Manages calculated data (e.g., visibility, UV coverage) and provides methods for validation, synchronization, and serialization. Maintains consistency between scans and their referenced entities, updating names and availability as entities are modified.

code

Unique identifier for the observation.

Type

`str`

observation_type

Type of observation, either 'VLBI' or 'SINGLE_DISH'.

Type

`str`

sources

Collection of source objects observed.

Type

`Sources`

telescopes

Collection of telescope objects used.

Type

`Telescopes`

frequencies

Collection of intermediate frequency (IF) objects.

Type*Frequencies***scans**

Collection of scan objects defining observation timing and targets.

Type*Scans***calculated_data**

Dictionary storing calculated results.

Type

Dict[str, Any]

isactive

Indicates whether the observation is active.

Type

bool

calculated_data: Dict[str, Any]**clear_calculated_data()**

Clear all cached calculation data for this observation.

code: str**copy()** → *Observation*

Create a deep copy of the Observation object.

frequencies: *Frequencies***classmethod from_dict(data: dict)** → *Observation*

Create an Observation object from a dictionary.

Restores `calculated_data` according to specified formats, handling `astropy.time.Time`, `numpy.ndarray`, and `astropy.units.Quantity` as needed.**Parameters****data** (*dict*) – Dictionary containing observation data.**Returns**

Deserialized Observation object.

Return type*Observation***Raises**

- **ValueError** – If critical data fields are missing or invalid.
- **TypeError** – If data structure is invalid.

get_calculated_data() → Dict[str, Any]

Retrieve all calculated data.

get_calculated_data_by_key(key: str) → Any

Retrieve calculated data for a specific key.

get_duration() → int | None

Retrieve the total observation duration in seconds by summing durations of active scans.

Returns

Total duration in seconds, or None if no active scans are found.

Return type

Optional[int]

`get_frequencies()` → *Frequencies*

Retrieve the Frequencies object.

`get_observation_code()` → str

Retrieve the observation code.

`get_observation_type()` → str

Retrieve the observation type.

`get_scans()` → *Scans*

Retrieve the Scans object.

`get_sources()` → *Sources*

Retrieve the Sources object.

`get_start_datetime()` → Time | None

Retrieve the earliest start time of active scans.

`get_telescopes()` → *Telescopes*

Retrieve the Telescopes object.

`name:` str`observation_type:` str`scans:` *Scans*`set_calculated_data_by_key(key: str, data: Any)` → None

Set calculated data for a specific key.

`sources:` *Sources*`telescopes:` *Telescopes*`to_dict()` → dict

Convert the Observation object to a dictionary for serialization.

Handles special serialization for `astropy.time.Time` arrays in `calculated_data['times']`, `numpy.ndarray` in `calculated_data['telescope_positions']`, and boolean arrays in `calculated_data['source_visibility']`, ensuring all data is JSON-serializable.

Raises**TypeError** – If an object in `calculated_data` cannot be serialized to JSON.`validate()` → bool

Validate the observation's data for consistency and completeness.

pastrocore.base.scans module

```
class pastrocore.base.scans.Scan (name: str = None, start: Time = None, duration: float = 1.0,
                                  source: Source | None = None, telescopes: List[Telescope |
                                  SpaceTelescope] = None, frequencies: List[IF] = None,
                                  is_off_source: bool = False, isactive: bool = True,
                                  observation: Observation = None)
```

Bases: *BaseEntity*

Base class representing a single observation scan with timing, source, telescopes, and frequencies.

`check_activity_status (observation: Observation) → bool`

`check_telescope_availability (observation: Observation, time: Time = None) → dict[str, bool]`

Check telescope availability for this scan at a given time.

`copy () → Scan`

Create a deep copy of the Scan object.

`duration: float`

`frequencies: List[IF]`

`classmethod from_dict (data: dict, observation: Observation = None) → Scan`

Create a Scan object from a dictionary, parsing ISO string to Time and resolving objects from Observation.

Ensures the source reference is synchronized with the observation and activity status is validated.

`get_MJD_endtime () → float`

Retrieve the end time in Modified Julian Date (MJD).

`get_MJD_starttime () → float`

Retrieve the start time in Modified Julian Date (MJD).

`get_duration () → float`

Retrieve the duration of the scan.

`get_end () → Time`

Retrieve the end time of the scan.

`get_frequencies (observation: Observation) → Frequencies`

Retrieve the frequencies associated with this scan.

`get_frequency_names () → List[str]`

Retrieve the list of frequency names.

`get_source (observation: Observation) → Source | None`

Retrieve the source associated with this scan.

`get_source_name () → str | None`

Retrieve the source name.

`get_start () → Time`

Retrieve the start time of the scan.

`get_telescope_names () → List[str]`

Retrieve the list of telescope names.

`get_telescopes (observation: Observation) → Telescopes`

Retrieve the telescopes associated with this scan.

`is_off_source: bool`

`name: str`

`set_duration (duration: float) → None`

Set the duration of the scan.

`set_frequencies (frequencies: List[IF], observation: Observation = None) → None`

Set the frequencies for the scan.

`set_source (source: Source | None, observation: Observation = None) → None`

Set the source for the scan.

`set_start (start: Time) → None`

Set the start time of the scan.

`set_telescopes (telescopes: List[Telescope | SpaceTelescope], observation: Observation = None) → None`

Set the telescopes for the scan.

`source: Source | None`

`start: Time`

`sync_with_observation (observation: Observation) → bool`

Synchronize scan attributes with the provided observation.

Updates source, telescopes, and frequencies to match references in the observation, validates the scan, and updates its activity status. If the source is not found, sets the scan to off-source and deactivates it.

Parameters

`observation (Observation)` – The observation to synchronize with.

Returns

True if synchronization is successful, False if critical validation fails.

Return type

bool

Raises

`TypeError` – If observation is not of type Observation.

`telescopes: List[Telescope | SpaceTelescope]`

`to_dict () → dict`

Convert the Scan object to a dictionary, serializing Time as ISO string.

`validate_with_observation (observation: Observation) → bool`

Validate scan attributes against an observation.

Ensures that telescopes and frequencies exist in the observation. For off-source scans, allows source to be None. Updates activity status based on observation context.

Parameters

`observation (Observation)` – The observation to validate against.

Returns

True if the scan is valid or can be made valid (e.g., off-source), False if critical validation fails.

Return type

bool

Raises

`TypeError` – If observation is not of type Observation.

`class pastrocore.base.scans.Scans (items: Dict[str, Scan] = None, name: str = None, isactive: bool = True, use_cache: bool = False)`

Bases: `BaseContainer[Scan]`

Base class representing a collection of Scan objects.

`activate_all (observation: Observation) → None`

Activate all scans in the collection that satisfy activity conditions.

`add(scan: Scan, observation: Observation = None) → None`

Add a Scan object to the collection with overlap checking.

`copy() → Scans`

Create a deep copy of the Scans object.

`create_scan(name: str = None, start: Time = None, duration: float = 1.0, source: Source | None = None, telescopes: List[Telescope | SpaceTelescope] = None, frequencies: List[F] = None, is_off_source: bool = False, isactive: bool = True, observation: Observation = None) → None`

Create and add a new Scan object to the collection.

`deactivate_all() → None`

Deactivate all scans in the collection.

`classmethod from_dict(data: dict, observation: Observation = None) → Scans`

Create a Scans object from a dictionary.

`get_active_scans(observation: Observation = None) → List[Scan]`

Retrieve all active scans, optionally filtering by entity activity in an Observation.

`get_inactive_scans() → List[Scan]`

Retrieve all inactive scans.

`isactive: bool`

`name: str`

`remove(name: str) → None`

Remove a scan by its name.

`set_scan(name: str, start: Time | None = None, duration: float | None = None, source: Source | None = None, telescopes: List[Telescope | SpaceTelescope] | None = None, frequencies: List[F] | None = None, is_off_source: bool | None = None, isactive: bool | None = None, observation: Observation = None) → None`

Update an existing Scan object in the collection with new parameters.

pastrocore.base.sources module

```
class pastrocore.base.sources.Source(name: str = 'SOURCE_DEFAULT', ra_h: float = 0.0,
                                     ra_m: float = 0.0, ra_s: float = 0.0, de_d: float = 0.0,
                                     de_m: float = 0.0, de_s: float = 0.0, name_J2000: str |
                                     None = None, alt_name: str | None = None, flux_table:
                                     Dict[float, float] | None = None, spectral_index: float |
                                     None = None, isactive: bool = True)
```

Bases: `BaseEntity`

Base class representing an astronomical source with coordinates, names, and optional flux properties.

`name`

Source name in B1950 notation.

Type

str

`ra_h`

Right Ascension hours (0-23).

Type

float

ra_m
Right Ascension minutes (0-59).
Type
float

ra_s
Right Ascension seconds (0-59.999).
Type
float

de_d
Declination degrees (-90 to 90).
Type
float

de_m
Declination minutes (0-59).
Type
float

de_s
Declination seconds (0-59.999).
Type
float

name_J2000
Source name in J2000 notation.
Type
Optional[str]

alt_name
Alternative source name.
Type
Optional[str]

flux_table
Flux table mapping frequencies (MHz) to flux values (Jy).
Type
Dict[float, float]

spectral_index
Spectral index for flux extrapolation.
Type
Optional[float]

isactive
Whether the source is active.
Type
bool

add_flux (*frequency: float, flux: float*) → None
Add a flux value for a specific frequency.

alt_name: str | None

```
clear_flux_table() → None
    Clear all entries from the flux table.

copy() → Source
    Create a deep copy of the Source object.

de_d: float

de_m: float

de_s: float

property dec_degrees: float
    Declination in decimal degrees.

flux_table: Dict[float, float]

classmethod from_dict(data: dict) → Source
    Create a Source instance from a dictionary, converting flux_table keys to float.

get_flux(frequency: float) → float | None
    Retrieve the flux for a given frequency, with interpolation or extrapolation.

name: str

name_J2000: str | None

property ra_degrees: float
    Right Ascension in decimal degrees.

ra_h: float

ra_m: float

ra_s: float

remove_flux(frequency: float) → None
    Remove a flux value for a specific frequency.

set_dec_degrees(dec_deg: float) → None
    Set Declination from decimal degrees.

set_ra_degrees(ra_deg: float) → None
    Set Right Ascension from decimal degrees.

spectral_index: float | None

class pastrocore.base.sources.Sources(items: Dict[str, Source] = None, name: str = None,
                                       isactive: bool = True, use_cache: bool = False)

    Bases: BaseContainer[Source]
    Base class representing a collection of Source objects.
    Manages a dictionary of astronomical sources indexed by their B1950 names, ensuring uniqueness and providing methods for querying and manipulation.

    _items
        Dictionary mapping source names to Source objects.

    Type
        Dict[str, Source]
```

isactive

Whether the Sources object itself is active.

Type

bool

activate_item(*name: str*) → None

Activate a specific source by its name.

Triggers synchronization with a parent Observation if present.

Parameters

name (*str*) – The name of the source to activate.

Raises

KeyError – If the name is not found in the collection.

copy() → *Sources*

Create a deep copy of the Sources object.

create_source(*name: str = 'SOURCE_DEFAULT', ra_h: float = 0.0, ra_m: float = 0.0, ra_s: float = 0.0, de_d: float = 0.0, de_m: float = 0.0, de_s: float = 0.0, name_J2000: str | None = None, alt_name: str | None = None, flux_table: Dict[float, float] | None = None, spectral_index: float | None = None, isactive: bool = True*) → None

Create and add a new Source object to the collection.

deactivate_item(*name: str*) → None

Deactivate a specific source by its name.

Triggers synchronization with a parent Observation if present.

Parameters

name (*str*) – The name of the source to deactivate.

Raises

KeyError – If the name is not found in the collection.

classmethod from_dict(*data: dict*) → *Sources*

Create a Sources object from a dictionary

isactive: bool

name: str

set_source(*name: str, ra_h: float | None = None, ra_m: float | None = None, ra_s: float | None = None, de_d: float | None = None, de_m: float | None = None, de_s: float | None = None, name_J2000: str | None = None, alt_name: str | None = None, flux_table: Dict[float, float] | None = None, spectral_index: float | None = None, isactive: bool | None = None*) → None

Update an existing Source object in the collection with provided parameters.

Parameters

- **name** (*str*) – Name of the source to update.
- **ra_h** (*Optional[float]*) – Right Ascension hours (0-23).
- **ra_m** (*Optional[float]*) – Right Ascension minutes (0-59).
- **ra_s** (*Optional[float]*) – Right Ascension seconds (0-59.999).
- **de_d** (*Optional[float]*) – Declination degrees (-90 to 90).
- **de_m** (*Optional[float]*) – Declination minutes (0-59).
- **de_s** (*Optional[float]*) – Declination seconds (0-59.999).

- **name_J2000** (*Optional[str]*) – Source name in J2000 notation.
- **alt_name** (*Optional[str]*) – Alternative source name.
- **flux_table** (*Optional[Dict[float, float]]*) – Flux table mapping frequencies (MHz) to flux values (Jy).
- **spectral_index** (*Optional[float]*) – Spectral index for flux extrapolation.
- **isactive** (*Optional[bool]*) – Whether the source is active.

Raises

- **KeyError** – If the source with the given name does not exist.
- **ValueError** – If provided parameters fail validation.

pastrocore.base.spacetelescope module

```
class pastrocore.base.spacetelescope.MountType(*values)
    Bases: Enum

    AZIMUTHAL = 'AZIM'

    EQUATORIAL = 'EQUA'

    SPACE = 'NONE'

class pastrocore.base.spacetelescope.SpaceTelescope(*, code: str = 'TS', name: str =
    'TEMPSPACETELESCOPE', type: str =
    'SpaceTelescope', orbit_file: str =
    'dummy_orbit.oem', diameter: float =
    1.0, sefd_table: Dict[float, float] | None
    = None, pitch_range: Tuple[float, float]
    = (-90.0, 90.0), yaw_range:
    Tuple[float, float] = (-180.0, 180.0),
    isactive: bool = True, use_kep: bool =
    True, kepler_elements: dict = None,
    interpolation_method: str =
    'chebyshev', surface_accuracy: float |
    None = None,
    surface_efficiency_table: Dict[float,
    float] | None = None,
    effective_area_table: Dict[float, float] |
    None = None,
    system_temperature_table: Dict[float,
    float] | None = None)
```

Bases: *Telescope*

Class representing a space-based telescope with orbital parameters and SEFD properties.

Inherits from *Telescope*, setting `mount_type` to 'NONE'. Stores orbit file path or Keplerian elements for position calculations, performed by *ScheduleCalculator*.

classmethod `from_dict(data: dict) → SpaceTelescope`

Create a *SpaceTelescope* object from a dictionary.

get_orbit() → str | None

Retrieve the orbit file path.

Returns

Path to the orbit file, or None if not set.

Return type

Optional[str]

`get_pitch_range()` → Tuple[float, float]

Retrieve the pitch range of the telescope.

`get_yaw_range()` → Tuple[float, float]

Retrieve the yaw range of the telescope.

`interpolation_method: str``kepler_elements: dict``orbit_file: str``pitch_range: Tuple[float, float]``set_interpolation_method(method: str)` → None

Set the interpolation method for orbit data.

`set_keplerian(a: float, e: float, i: float, raan: float, argp: float, nu: float, epoch: Time, mu: float = 398600441800000.0)` → None

Set Keplerian elements for orbit calculation.

`set_orbit(orbit_file: str)` → None

Set the orbit file path.

Parameters`orbit_file (str)` – Path to the orbit file.**Raises**

- **TypeError** – If `orbit_file` is not a string.
- **ValueError** – If `orbit_file` is empty.

`to_dict()` → dict

Convert the SpaceTelescope object to a dictionary for serialization.

`type: str``use_kep: bool``yaw_range: Tuple[float, float]`**pastrocore.base.telescope module**`class pastrocore.base.telescope.MountType(*values)``Bases: Enum``AZIMUTHAL = 'AZIM'``EQUATORIAL = 'EQUA'``SPACE = 'NONE'`

```
class pastrocore.base.telescope.Telescope(*, code: str = 'TT', name: str =
    'TEMPTELESCOPE', type='Telescope', x: float =
    0.0, y: float = 0.0, z: float = 0.0, vx: float = 0.0, vy:
    float = 0.0, vz: float = 0.0, diameter: float = 1.0,
    sefd_table: Dict[float, float] | None = None,
    elevation_range: Tuple[float, float] = (15.0, 90.0),
    azimuth_range: Tuple[float, float] = (0.0, 360.0),
    mount_type: str = 'AZIM', isactive: bool = True,
    surface_accuracy: float | None = None,
    surface_efficiency_table: Dict[float, float] | None =
    None, effective_area_table: Dict[float, float] | None =
    None, system_temperature_table: Dict[float,
    float] | None = None)
```

Bases: [BaseEntity](#)

Class representing a ground-based telescope with ITRF coordinates, velocities, and SEFD properties.

`add_sefd(frequency: float, sefd: float) → None`

Add an SEFD value for a specific frequency to the SEFD table.

`azimuth_range: Tuple[float, float]`

`calculate_effective_area(frequency: float) → float | None`

Calculate effective area and add to table.

`calculate_sefd(frequency: float) → float | None`

Calculate SEFD from effective area and system temperature, add to table.

`calculate_surface_efficiency(frequency: float) → float | None`

Calculate surface efficiency using Ruze formula and add to table.

`clear_sefd_table() → None`

Clear all entries from the SEFD table.

`code: str`

`copy() → Telescope`

Create a deep copy of the Telescope object.

`diameter: float`

`effective_area_table: Dict[float, float]`

`elevation_range: Tuple[float, float]`

`classmethod from_dict(data: dict) → Telescope`

Create a Telescope object from a dictionary.

`get_azimuth_range() → Tuple[float, float]`

Return the telescope's azimuth range.

Returns

The minimum and maximum azimuth angles in degrees.

Return type

Tuple[float, float]

`get_code() → str`

Return the telescope's code.

Returns

The unique code of the telescope.

Return type

str

`get_coordinates()` → Tuple[float, float, float]

Return the telescope's ITRF coordinates.

Returns

The (x, y, z) coordinates in meters.

Return type

Tuple[float, float, float]

`get_effective_area(frequency: float)` → float | None

Retrieve the effective area for a given frequency, with linear interpolation.

`get_elevation_range()` → Tuple[float, float]

Return the telescope's elevation range.

Returns

The minimum and maximum elevation angles in degrees.

Return type

Tuple[float, float]

`get_sefd(frequency: float)` → float | None

Retrieve the SEFD for a given frequency, with linear interpolation if needed.

`get_surface_efficiency(frequency: float)` → float | None

Retrieve the surface efficiency for a given frequency, with linear interpolation.

`get_system_temperature(frequency: float)` → float | None

Retrieve the system temperature for a given frequency, with linear interpolation.

`get_velocities()` → Tuple[float, float, float]

Return the telescope's ITRF velocities.

Returns

The (vx, vy, vz) velocities in meters.

Return type

Tuple[float, float, float]

`mount_type`: *MountType*`name`: str`remove_sefd(frequency: float)` → None

Remove an SEFD value for a specific frequency from the SEFD table.

`sefd_table`: Dict[float, float]`set(params: Dict[str, Any])` → None

Set entity attributes from a dictionary with type validation, handling mount_type.

`surface_accuracy`: float | None`surface_efficiency_table`: Dict[float, float]`system_temperature_table`: Dict[float, float]`to_dict()` → dict

Convert the Telescope object to a dictionary for serialization.

`type`: str

```
vx: float
vy: float
vz: float
x: float
y: float
z: float
```

pastrocore.base.telescopes module

```
class pastrocore.base.telescopes.Telescopes (items: Dict[str, Telescope | SpaceTelescope] |
                                             None = None, name: str = None, isactive: bool =
                                             True, use_cache: bool = False)
```

Bases: `BaseContainer`[`Telescope` | `SpaceTelescope`]

Class representing a collection of Telescope and SpaceTelescope objects.

Manages a dictionary of telescopes, indexed by their code, ensuring no duplicates. Inherits from BaseContainer for collection management, activation/deactivation, and serialization. Supports synchronization with a parent Observation object.

_items

Dictionary of telescope objects, keyed by code.

Type

Dict[str, `Telescope` | `SpaceTelescope`]

isactive

Whether the Telescopes object itself is active. Inherited from BaseEntity.

Type

bool

Notes

- Telescopes are identified by their unique code (`get_code()`), used as the dictionary key.
- Activation/deactivation triggers synchronization with a parent Observation via `_parent._sync_scans_with_activation`.
- Both *name* and *code* must be unique and valid strings (no spaces or special characters).
- Transition from index-based to code-based access requires updating dependent code to use telescope codes.

Examples

```
>>> tels = Telescopes()
>>> tels.create_telescope(code="RT32", name="Radio Telescope 32m", diameter=32.
→0)
>>> print(tels)
Telescopes(count=1, active=1, inactive=0)
>>> tels.add(Telescope(code="RT32", name="Duplicate"))
Traceback (most recent call last):
...
ValueError: Telescope with code 'RT32' already exists in Telescopes
```


`activate_item(name: str) → None`

Activate a specific telescope by its name.

Triggers synchronization with a parent Observation if present.

Parameters

`name (str)` – The name of the telescope to activate.

Raises

KeyError – If the name is not found in the collection.

`copy()` → *Telescopes*

Create a deep copy of the Telescopes object.

`create_telescope(code: str = 'TEMP', name: str = 'Temporary Telescope', x: float = 0.0, y: float = 0.0, z: float = 0.0, vx: float = 0.0, vy: float = 0.0, vz: float = 0.0, diameter: float = 1.0, sefd_table: Dict[float, float] | None = None, elevation_range: Tuple[float, float] = (15.0, 90.0), azimuth_range: Tuple[float, float] = (0.0, 360.0), mount_type: str = 'AZIM', isactive: bool = True) → None`

Create and add a new ground-based Telescope object to the collection.

Parameters

- `code (str)` – Unique short name. Defaults to “TEMP”.
- `name (str)` – Full name (set to code for consistency). Defaults to “Temporary Telescope”.
- `x (float)` – X-coordinate in ITRF (meters). Defaults to 0.0.
- `y (float)` – Y-coordinate in ITRF (meters). Defaults to 0.0.
- `z (float)` – Z-coordinate in ITRF (meters). Defaults to 0.0.
- `vx (float)` – X-velocity in ITRF (meters/year). Defaults to 0.0.
- `vy (float)` – Y-velocity in ITRF (meters/year). Defaults to 0.0.
- `vz (float)` – Z-velocity in ITRF (meters/year). Defaults to 0.0.
- `diameter (float)` – Antenna diameter in meters. Defaults to 1.0.
- `sefd_table (Optional[Dict[float, float]])` – SEFD table (MHz: Jy). Defaults to None.
- `elevation_range (Tuple[float, float])` – Min and max elevation in degrees. Defaults to (15.0, 90.0).
- `azimuth_range (Tuple[float, float])` – Min and max azimuth in degrees. Defaults to (0.0, 360.0).
- `mount_type (str)` – Mount type ('EQUA', 'AZIM', or 'NONE'). Defaults to “AZIM”.
- `isactive (bool)` – Whether the telescope is active. Defaults to True.

Raises

- **TypeError** – If inputs are of incorrect type.
- **ValueError** – If code or name is a duplicate, invalid, or other Telescope initialization errors occur.

`deactivate_item(name: str) → None`

Deactivate a specific telescope by its name.

Triggers synchronization with a parent Observation if present.

Parameters

`name (str)` – The name of the telescope to deactivate.

Raises

KeyError – If the name is not found in the collection.

classmethod `from_dict(data: Dict[str, Any]) → Telescopes`

Create a Telescopes object from a dictionary.

Ensures that both name and code of telescopes are unique during deserialization.

Parameters

data (`Dict[str, Any]`) – Dictionary containing telescope data.

Returns

A new Telescopes object populated with deserialized telescopes.

Return type

Telescopes

Raises

ValueError – If duplicate name or code is found, or if telescope data is invalid.

isactive: bool

name: str

1.1.1.2 Module contents

1.1.2 pastrocore.super package

1.1.2.1 Submodules

pastrocore.super.schedule_calculator module

class `pastrocore.super.schedule_calculator.ScheduleCalculator` (*manipulator*: Manipulator)

Bases: *Super*

Scheduler implementation of Calculator for performing astronomical scheduling calculations.

Provides methods to calculate telescope positions, source visibility, UV coverage, sun angles, and more for Observations and Projects. Supports caching of results and multi-threaded execution for efficiency.

manipulator

The Manipulator instance used to manage object interactions.

_lock

Thread lock for safe data caching.

Type

`threading.Lock`

Examples

```
>>> from unit_scheduling.super.manipulator import ScheduleManipulator
>>> manipulator = ScheduleManipulator()
>>> calculator = ScheduleCalculator(manipulator)
>>> obs = Observation()
>>> result = calculator.calculate(obs, {"store_key": "uv_coverage_f0", "freq_
↪name": 0})
>>> print(result)
{'0': {'uv_points': {...}}}
```

`pastrocore.super.schedule_calculator.time_execution(func)`

Decorator to measure and log the execution time of calculation methods.

Parameters

`func` – The function to decorate.

Returns

Wrapped function that logs execution duration.

Return type

Callable

pastrocore.super.schedule_configurator module

`class pastrocore.super.schedule_configurator.ScheduleConfigurator (manipulator: Manipulator)`

Bases: *Super*

Implementation of Configurator for configuring scheduling entities using the Super framework.

Provides methods to configure astronomical scheduling entities with validation and nested configuration support. Integrates with Manipulator for method validation. Returns the result of the final method called on the object (e.g., `get`, `get_code`) as the result in the response dictionary.

Parameters

`manipulator` – The Manipulator instance for method lookup and validation.

Returns

A dictionary with results of the configuration operation, managed by `Super.execute`.

Return type

`Dict[str, Any]`

pastrocore.super.schedule_inspector module

`class pastrocore.super.schedule_inspector.ScheduleInspector (manipulator: Manipulator)`

Bases: *Super*

Implementation of Inspector for inspecting scheduling entities using the Super framework.

Provides methods to inspect astronomical scheduling entities (IF, Frequencies, Source, Sources, Telescope, SpaceTelescope, Telescopes, Scan, Scans, Observation, ScheduleProject) by invoking their getter methods. Returns the result of a final method (e.g., `get`, `get_code`) in the response dictionary.

Parameters

`manipulator` – The Manipulator instance for method lookup and validation.

Returns

Dictionary containing the result of the final method, wrapped by `Super.execute` in a response dictionary with keys `status`, `object`, `method`, `result`, and `error` (if `status=False`).

Return type

`Dict[str, Any]`

Examples

```

>>> from pastrocore.super.manipulator import ScheduleManipulator
>>> manipulator = ScheduleManipulator()
>>> inspector = ScheduleInspector(manipulator)
>>> source = Source(name="3C 286")
>>> result = inspector.execute(source, {"get": "name"})
{'status': True, 'object': <Source>, 'method': '_inspect_source', 'result': {
↳ 'name': '3C 286'}}
>>> project = ScheduleProject()
>>> project.create_item(item_code="OBS001")
>>> result = inspector.execute(project, {"name": "OBS001", "get": "code"})
{'status': True, 'object': <Observation>, 'method': '_inspect_observation',
↳ 'result': 'OBS001'}

```

pastrocore.super.schedule_manipulator module

class pastrocore.super.schedule_manipulator.**ScheduleManipulator** (*project:*
ScheduleProject | None
= None)

Bases: *Manipulator*

Scheduler implementation of Manipulator for managing astronomical scheduling operations.

Extends the base Manipulator class to provide a centralized interface for configuring, inspecting, calculating, and visualizing ScheduleProject and its components. Registers default operations (configure, inspect, calculate, visualize) with corresponding handler classes.

Parameters

project (*Optional*[ScheduleProject]) – The ScheduleProject instance to manage. Defaults to None.

managing_object

The object being managed (typically a ScheduleProject).

base_classes

List of supported base classes for method validation.

Type

List[type]

operations

Registered operations (e.g., “configure”, “inspect”).

Type

Dict[str, Any]

Examples

```

>>> from unit_scheduling.super.schedule_project import ScheduleProject
>>> manipulator = ScheduleManipulator(project=ScheduleProject(name="TestProject"
↳ ))
>>> manipulator.operations["configure"]
<ScheduleConfigurator object at ...>
>>> manipulator.get_methods_for_type(Source)
{'get_name': <function ...>, 'set_name': <function ...>, ...}

```

pastrocore.super.schedule_project module

```
class pastrocore.super.schedule_project.ScheduleProject (name: str =
                                                    'OBS_DEFAULT_PROJECT',
                                                    items: Dict[str, Observation] |
                                                    None = None)
```

Bases: *Project*

Container for managing multiple observations, inheriting from Project.

Represents a project that organizes a collection of Observation objects, indexed by their observation names. Provides methods to add, create, set, retrieve, and manage observations, as well as serialize/deserialize the project.

Examples

```
>>> project = ScheduleProject(name="MyProject")
>>> project.create_item(item_name="OBS001")
>>> project.get_observation("OBS001").get_observation_code()
'OBS001'
>>> project.set_item("OBS001", Observation(name="OBS001", isactive=False))
>>> project.get_observation("OBS001").isactive
False
>>> project.set_project(name="NewProject", items={})
>>> project.get_project()["name"]
'NewProject'
```

add_item (*item*: *Observation*) → None

Add an observation to the project.

Parameters

item (*Observation*) – The Observation object to add.

Raises

TypeError – If the item is not an Observation object.

clear_calculated_data ()

Clear all cached calculation data for this project and its observations.

create_item (*item_code*: *str* = 'OBS_DEFAULT', *isactive*: *bool* = True, *observation_type*: *str* = 'VLBI') → None

Create and add a new Observation object to the project.

Parameters

- **item_code** (*str*) – The code for the new observation. Defaults to "OBS_DEFAULT".
- **isactive** (*bool*) – Whether the new observation is active. Defaults to True.
- **observation_type** (*str*) – The type of observation ('VLBI' or 'SINGLE_DISH'). Defaults to "VLBI".

Raises

ValueError – If *item_code* is not a non-empty string or *observation_type* is invalid.

classmethod from_dict (*data*: *Dict[str, Any]*) → *ScheduleProject*

Create a ScheduleProject from a dictionary.

Supports both new format (with 'items') and legacy format (with 'observations').

Parameters

`data` (*Dict[str, Any]*) – A dictionary containing 'name' and either 'items' or 'observations'.

Returns

A new `ScheduleProject` instance populated with the dictionary data.

Return type

ScheduleProject

Raises

`ValueError` – If the data is invalid or cannot be deserialized.

`get_observation(name: str) → Observation`

Retrieve an observation by its name.

Parameters

`name` (*str*) – The name of the observation to retrieve.

Returns

The `Observation` object with the specified name.

Return type

Observation

Raises

`KeyError` – If the observation name is not found.

`get_observation_by_code(code: str) → Observation`

Retrieve an observation by its code.

Parameters

`code` (*str*) – The code of the observation to retrieve.

Returns

The `Observation` object with the specified code.

Return type

Observation

Raises

`ValueError` – If the code is not a non-empty string.

`get_project() → Dict[str, Any]`

Get the entire project configuration as a dictionary.

Returns

A dictionary containing the project name and a list of observation dictionaries.

Return type

Dict[str, Any]

`name: str`

`set_item(name: str, item: Observation) → None`

Set or replace an observation in the project by its name.

Parameters

- `name` (*str*) – The name of the observation to set.
- `item` (*Observation*) – The `Observation` object to set.

Raises

`TypeError` – If the item is not an `Observation` object.

`set_project (name: str, items: Dict[str, Observation]) → None`

Set the entire project configuration, replacing name and observations.

Parameters

- **name** (*str*) – The new name for the project. Must be a non-empty string.
- **items** (*Dict[str, Observation]*) – The new dictionary of observations, keyed by observation name.

Raises

- **TypeError** – If any item in items is not an Observation object.
- **ValueError** – If name is not a non-empty string.

`to_dict () → Dict[str, Any]`

Convert ScheduleProject to a dictionary for serialization.

Returns

A dictionary with the project name and observations.

Return type

Dict[str, Any]

pastrocore.super.schedule_visualizer module

`class pastrocore.super.schedule_visualizer.ScheduleVisualizer (manipulator: Manipulator)`

Bases: *Super*

EARTH_DIAMETER: float = 12742000.0

SPEED_OF_LIGHT: float = 299792458.0

execute (*obj: ScheduleProject | Observation | Telescope | SpaceTelescope | Telescopes | Source | Sources | Scan | Scans | IF | Frequencies*, *attributes: Dict[str, Any] = None*, *method: str = None*) → Dict[str, Any]

Execute visualization operation on the specified object.

`get_style_config () → Dict[str, Any]`

Return the current style configuration.

Returns

A copy of the current style configuration dictionary.

Return type

Dict[str, Any]

`set_style_config (config: Dict[str, Any], partial: bool = False) → None`

Set a new style configuration and apply it.

Parameters

- **config** (*Dict[str, Any]*) – New style configuration dictionary or partial updates.
- **partial** (*bool*) – If True, update only provided keys; if False, replace entire config.

Raises

ValueError – If the provided configuration is invalid.

1.1.2.2 Module contents

1.1.3 pastrocore.utils package

1.1.3.1 Submodules

pastrocore.utils.catalogmanager module

```
class pastrocore.utils.catalogmanager.CatalogManager (source_file: str | None = None,  
                                                    telescope_file: str | None = None)
```

Bases: object

Manages catalogs of astronomical sources and telescopes.

Loads and stores source and telescope data from text files, providing methods to query and filter the catalogs. Supports B1950/J2000 source names, RA/DEC ranges, and telescope codes/types.

source_catalog

Collection of Source objects.

Type

Sources

telescope_catalog

Collection of Telescope objects.

Type

Telescopes

Notes

- Logging is integrated via *common.utils.logging_setup.logger*.
- Source file format: *name j2000_name alt_name ra_hh:mm:ss.ssss dec_dd:mm:ss.ssss*.
- Telescope file format: *number short_name full_name x y z diameter*.
- Lines starting with '#' or empty lines are skipped during loading.

Examples

```
>>> cm = CatalogManager(source_file="sources.txt", telescope_file="telescopes.  
↪txt")  
>>> source = cm.get_source("3C273")  
>>> telescopes = cm.get_telescopes_by_type("Telescope")  
>>> print(cm)  
CatalogManager(sources=<num>, telescopes=<num>)
```

clear_catalogs() → None

Clear both the source and telescope catalogs.

Notes

- Resets source_catalog and telescope_catalog to empty collections.

`get_source(name: str) → Source | None`

Retrieve a source from the catalog by its B1950 or J2000 name.

Parameters

`name (str)` – The B1950 or J2000 name of the source.

Returns

The matching Source object, or None if not found.

Return type

Optional[Source]

`get_sources_by_dec_range(dec_min: float, dec_max: float) → List[Source]`

Retrieve sources within a specified declination (DEC) range in degrees.

Parameters

- `dec_min (float)` – Minimum DEC in degrees.
- `dec_max (float)` – Maximum DEC in degrees.

Returns

List of Source objects within the DEC range.

Return type

List[Source]

`get_sources_by_ra_range(ra_min: float, ra_max: float) → List[Source]`

Retrieve sources within a specified right ascension (RA) range in degrees.

Parameters

- `ra_min (float)` – Minimum RA in degrees.
- `ra_max (float)` – Maximum RA in degrees.

Returns

List of Source objects within the RA range.

Return type

List[Source]

`get_telescope(code: str) → Telescope | None`

Retrieve a telescope from the catalog by its code.

Parameters

`code (str)` – The unique code of the telescope.

Returns

The matching Telescope object, or None if not found.

Return type

Optional[Telescope]

`get_telescopes_by_type(telescope_type: str = 'Telescope') → List[Telescope]`

Retrieve telescopes filtered by type.

Parameters

`telescope_type (str)` – The type of telescope to filter by (currently only “Telescope” is supported). Defaults to “Telescope”.

Returns

List of Telescope objects matching the specified type.

Return type

List[Telescope]

`load_source_catalog(source_file: str) → None`

Load a sources catalog from a text file into the `source_catalog` attribute.

Expected format: *name j2000_name alt_name ra_hh:mm:ss.ssss dec_dd:mm:ss.ssss*.

Parameters

`source_file (str)` – Path to the sources catalog file.

Raises

- **FileNotFoundError** – If the file does not exist.
- **ValueError** – If there is an error parsing the catalog data (e.g., invalid RA/DEC format).

Notes

- Logs warnings for invalid lines and a summary of loaded/failed sources.

`load_telescope_catalog(telescope_file: str) → None`

Load a telescopes catalog from a text file into the `telescope_catalog` attribute.

Expected format: *number short_name full_name x y z diameter*.

Parameters

`telescope_file (str)` – Path to the telescopes catalog file.

Raises

- **FileNotFoundError** – If the file does not exist.
- **ValueError** – If there is an error parsing the catalog data (e.g., invalid numeric values).

Notes

- Logs warnings for invalid lines and a summary of loaded/failed telescopes.
- Velocities (`vx`, `vy`, `vz`) are set to 0.0 as they are not provided in the catalog format.

1.1.3.2 Module contents

1.2 Module contents

COMMON PACKAGE

2.1 Subpackages

2.1.1 common.base package

2.1.1.1 Submodules

common.base.basecontainer module

```
class common.base.basecontainer.BaseContainer(items: Dict[str, T] = None, name: str = None,  
                                              isactive: bool = True, use_cache: bool =  
                                              False)
```

Bases: `BaseEntity`, `ABC`, `Generic[T]`

Abstract base class for managing collections of `BaseEntity` objects using a dictionary.

Provides a foundation for container classes in the MBS system. Manages a collection of entities indexed by their *name* attribute, with support for validation, activation state management, and universal serialization. Subclasses can extend validation logic or add specialized behavior.

_items

Dictionary mapping entity names to their instances.

Type

`Dict[str, T]`

_fields

Inherited from `BaseEntity`, contains type annotations including *_items*.

Type

`Dict[str, type]`

_use_cache

Flag to enable caching for *to_dict* results.

Type

`bool`

_cached_to_dict

Cached result of *to_dict* to improve performance.

Type

`dict, optional`

Notes

- Logging is integrated via `common.utils.logging_setup.logger` to track operations.
- This is an abstract base class and cannot be instantiated directly; it must be subclassed.
- The `name` attribute of contained entities is used as the key, ensuring uniqueness within the container.
- Serialization methods `to_dict` and `from_dict` handle the entire collection, including nested entities.
- Optional caching in `to_dict` can be enabled by setting `_use_cache=True`.
- Direct modification of `_items` bypasses validation and cache invalidation. Use `add`, `remove`, or `set_items` instead.

Examples

```
>>> class MyItem(BaseEntity):
...     value: int
>>> class MyContainer(BaseContainer[MyItem]):
...     pass
>>> container = MyContainer(name="test_container")
>>> item = MyItem(name="item1", value=42)
>>> container.add(item)
>>> print(container.to_dict())
{'name': 'test_container', 'isactive': True, 'items': {'item1': {'name': 'item1',
→ 'isactive': True, 'value': 42}}}
>>> new_container = MyContainer.from_dict({'name': 'test_container', 'isactive'
→ ': True, 'items': {'item1': {'name': 'item1', 'isactive': True, 'value': 42}}
→ })
>>> print(container.get_items())
[MyItem(name='item1', isactive=True, value=42)]
```

`activate_all()` → None

Activate all items in the container.

Raises

ValueError – If the container is empty.

`activate_item(name: str)` → None

Activate an item in the container by its name.

Parameters

name (*str*) – The name of the item to activate.

`add(item: T)` → None

Add an item to the collection using its name as the key.

Parameters

item (*T*) – The item to add to the container.

Raises

- **ValueError** – If the item's name is None or already exists in the container.
- **TypeError** – If the item's type does not match the expected type T.

`clear()` → None

Remove all items from the container.

Notes

- Logs an info message indicating the container has been cleared.

`clone(deep: bool = True) → BaseContainer[T]`

Create a deep copy of the container.

Returns

A new instance of the same class with identical items.

Return type

`BaseContainer[T]`

`deactivate_all() → None`

Deactivate all items in the container.

Raises

`ValueError` – If the container is empty.

`deactivate_item(name: str) → None`

Deactivate an item in the container by its name.

Parameters

`name (str)` – The name of the item to deactivate.

`drop_active() → None`

Remove all active items from the container.

Raises

`ValueError` – If there are no active items.

`drop_inactive() → None`

Remove all inactive items from the container.

Raises

`ValueError` – If there are no inactive items.

`classmethod from_dict(data: dict) → BaseContainer`

Create a container instance from a dictionary.

Reconstructs a container instance from serialized data, including its name, activation status, and all items, with nested entities recursively deserialized. Handles Union types for items.

Parameters

`data (dict)` – Dictionary containing the container's serialized data, typically from `to_dict`.

Returns

A new instance of the subclass initialized with the dictionary data.

Return type

`BaseContainer`

Raises

- `TypeError` – If the item type cannot be resolved or if data is invalid.
- `ValueError` – If item data cannot be mapped to a valid type in a Union.

`get(name: str) → T | None`

Retrieve an item from the container by its name.

Parameters

`name (str)` – The name of the item to retrieve.

Returns

The item associated with the specified name, or None if not found.

Return type
Optional[T]

Notes

- Logs a warning if the item is not found, rather than raising an exception.

`get_active_items()` → List[T]

Retrieve all active items in the container.

Returns
A list of items where isactive is True.

Return type
List[T]

`get_all()` → Dict[str, T]

Retrieve all items in the container with their names as keys.

Returns
the items dictionary, mapping names to entities.

Return type
Dict[str, T]

`get_by_value(conditions: Dict[str, Any])` → List[T]

Retrieve items from the container where all specified attributes match the given values.

Parameters
`conditions` (Dict[str, Any]) – A dictionary where keys are attribute names and values are the desired values (e.g., {"frequency": 1000.0, "isactive": False}). If empty, returns all items.

Returns
A list of items where all specified attributes equal the given values.

Return type
List[T]

Raises
AttributeError – If any specified attribute does not exist in the items.

`get_inactive_items()` → List[T]

Retrieve all inactive items in the container.

Returns
A list of items where isactive is False.

Return type
List[T]

`get_items()` → List[T]

Retrieve all items in the container as a list, without their names.

Returns
A list of all items in the container.

Return type
List[T]

`has_item(name: str)` → bool

Check if an item with the specified name exists in the container.

Parameters
`name` (str) – The name of the item to check.

Returns

True if the item exists, False otherwise.

Return type

bool

`property items: Dict[str, T]`

Read-only access to the items dictionary.

`name: str`

`remove(name: str) → None`

Remove an item from the container by its name.

Parameters

`name (str)` – The name of the item to remove.

`set(params: Dict[str, Any]) → None`

Set container attributes from a dictionary with type validation.

Parameters

`params (Dict[str, Any])` – Dictionary with attribute names and values to update.

Raises

- **ValueError** – If an attribute is not defined in the class annotations.
- **TypeError** – If an attribute value does not match its annotated type.

`set_item(name: str, item: T) → None`

Set or replace an item in the container by its name.

Parameters

- `name (str)` – The name of the item to set.
- `item (T)` – The item to add or replace.

Raises

- **ValueError** – If the item's name does not match the provided name or if it fails validation.
- **TypeError** – If the item's type does not match the expected type T.

`set_items(items: Dict[str, T]) → None`

Set or replace all items in the container.

Parameters

`items (Dict[str, T])` – Dictionary of items to set.

Raises

ValueError – If any item fails validation or has a mismatched name.

`to_dict(handle_cyclic_refs: str = 'mark') → dict`

Convert the container to a dictionary for serialization.

Serializes the container's state, including its name, activation status, and all items, with nested entities recursively serialized. Uses caching if enabled.

Parameters

`handle_cyclic_refs (str)` – How to handle cyclic references. Options: - "mark": Replace with "<cyclic reference>" (default). - "ignore": Skip cyclic references. - "raise": Raise an error on cyclic references.

Returns

A dictionary containing the container's serialized data.

Return type

dict

Raises

ValueError – If `handle_cyclic_refs` is invalid or cyclic reference is detected with “raise” option.

common.base.baseentity module

```
class common.base.baseentity.BaseEntity(*, name: str, isactive: bool = True, use_cache: bool = False, **kwargs)
```

Bases: ABC

Abstract base class for entities with attribute management, type validation, and universal serialization.

Provides a foundation for base entity classes in the MBS system. Defines common functionality for managing attributes with type checking, an active/inactive state, and universal serialization methods, including support for nested entities.

name

An optional identifier for the entity.

Type

str, optional

isactive

Indicates whether the entity is active or inactive.

Type

bool

_fields

Class-level mapping of attribute names to their expected types (from annotations).

Type

Dict[str, type]

Notes

- Logging is integrated via `common.utils.logging_setup.logger` to track initialization and state changes.
- This is an abstract base class and cannot be instantiated directly; it must be subclassed.
- Attributes are validated against type annotations defined in `__annotations__`.
- Serialization methods `to_dict` and `from_dict` automatically handle all annotated attributes, including nested entities.

Examples

```
>>> class NestedEntity(BaseEntity):
...     value: int
>>> class MyEntity(BaseEntity):
...     name: str
...     nested: NestedEntity
>>> nested = NestedEntity(value=42)
>>> entity = MyEntity(name="test", isactive=True, nested=nested)
```

(continues on next page)

(continued from previous page)

```

>>> print(entity.to_dict())
{'name': 'test', 'isactive': True, 'nested': {'name': None, 'isactive': True,
→ 'value': 42}}
>>> new_entity = MyEntity.from_dict({'name': 'test', 'isactive': True, 'nested
→ ': {'name': None, 'isactive': True, 'value': 42}})
>>> print(new_entity)
MyEntity(name='test', isactive=True, nested=NestedEntity(isactive=True,
→ value=42))

```

activate() → None

Activate the entity, setting its status to active.

Notes

- Logs an info message indicating the entity has been activated.

clone() → *BaseEntity*

Create a deep copy of the entity.

Returns

A new instance of the same class with identical attributes.

Return type

*BaseEntity***deactivate()** → None

Deactivate the entity, setting its status to inactive.

Notes

- Logs an info message indicating the entity has been deactivated.

classmethod from_dict(data: dict) → *BaseEntity*

Create an entity instance from a dictionary.

Automatically reconstructs an entity instance from serialized data, ignoring the 'type' field, and setting its name, activation status, and annotated attributes, including nested entities.

Parameters

data (*dict*) – Dictionary containing the entity's serialized data, typically from *to_dict*.

Returns

A new instance of the subclass initialized with the dictionary data.

Return type

BaseEntity

Raises

- **TypeError** – If a value in the dictionary does not match the annotated type.
- **ValueError** – If an unknown attribute is provided in the dictionary.

get(key: str | List[str] | None = None) → Any | Dict[str, Any]

Retrieve one or more attributes of the entity.

Parameters

key (*Union[str, List[str], None]*, *optional*) – The name of a single attribute, a list of attribute names, or None. If a string, returns the attribute's value.

If a list of strings, returns a dictionary with the specified attributes. If None, returns a dictionary of all public attributes. Defaults to None.

Returns

The value of the specified attribute if key is a string,
a dictionary of requested attributes if key is a list, or a dictionary of all public attributes if key is None.

Return type

Union[Any, Dict[str, Any]]

Raises

KeyError – If any specified key is not found in the entity's annotated fields.

Examples

```
>>> if_obj = IF(name="IF1", frequency=1000.0, bandwidth=16.0)
>>> if_obj.get("frequency")
1000.0
>>> if_obj.get(["frequency", "bandwidth"])
{'frequency': 1000.0, 'bandwidth': 16.0}
>>> if_obj.get()
{'name': 'IF1', 'isactive': True, 'frequency': 1000.0, 'bandwidth': 16.0,
↪ 'polarizations': []}
```

has_attribute(key: str) → bool

Check if the entity has a specific attribute.

Parameters

key (str) – The name of the attribute to check.

Returns

True if the attribute exists in the entity's fields and is set, False otherwise.

Return type

bool

isactive: bool

name: str

set(params: Dict[str, Any]) → None

Set entity attributes from a dictionary with type validation.

Parameters

params (dict) – Dictionary with attribute names and values to update.

Raises

- **TypeError** – If an attribute value does not match its annotated type.
- **ValueError** – If an attribute is not defined in the class annotations.

Notes

- Only attributes defined in `__annotations__` can be set.
- Logs an info message with updated attributes.

`to_dict()` → dict

Convert the entity to a dictionary for serialization.

Automatically serializes the entity's state, including all annotated attributes, with nested entities recursively serialized. Always includes a 'type' field with the class name.

Returns

A dictionary containing the entity's serialized data.

Return type

dict

class `common.base.baseentity.EntityMeta` (*name, bases, attrs*)

Bases: ABCMeta

Metaclass for BaseEntity to handle type annotations and enforce attribute validation.

Automatically collects type annotations from the subclass and configures the entity to validate attributes against these types during initialization and updates.

`_fields`

Dictionary of annotated attribute names and their expected types.

Type

Dict[str, type]

2.1.1.2 Module contents

2.1.2 common.super package

2.1.2.1 Submodules

common.super.manipulator module

class `common.super.manipulator.Manipulator` (*managing_object: Any | None = None, base_classes: List[Type] | None = None, operations: Dict[str, Callable] | None = None, strict_type_check: bool = False*)

Bases: ABC

Abstract class for managing and processing operations on objects.

Provides a framework for registering operations and their associated methods, managing a central object, and processing requests. Maintains a registry of supported object types and their methods, with caching for performance. Subclasses can extend this to implement specific manipulation logic.

`_managing_object`

The central object being managed.

Type

Optional[Any]

`_base_classes`

List of base classes whose methods are registered.

Type

List[Type]

`_operations`

Dictionary mapping operation names to super-instance handlers.

Type

Dict[str, Callable]

_registry

Registry of object types and their available methods.

Type

Dict[Type, Dict[str, Callable]]

Notes

- Uses *functools.lru_cache* to optimize method registry generation.
- Logging is integrated via *common.utils.logging_setup.logger*.
- Operations are executed via super-instances that must have an *execute* method.
- Results are returned as dictionaries with keys: status (bool), object (Any), method (str | None), result (Any), error (str | None, included only if status=False).

Examples

```
>>> manip = Manipulator(base_classes=[list])
>>> manip.register_operation("append", Super()) # Assuming super-class with
↳execute method
>>> manip.process_request({"operation": "append", "obj": [], "attributes": {
↳"value": 1}})
{"status": True, "object": [1], "method": "append", "result": True}
```

get_managing_object() → Any | None

Retrieve the central managing object.

Returns

The managing object, or None if not set.

Return type

Optional[Any]

get_methods_for_type(obj_type: Type) → Dict[str, Callable]

Retrieve the registered methods for a given object type.

Parameters**obj_type** (*Type*) – The type of object to query methods for.**Returns**

Dictionary of method names and their callable implementations.

Return type

Dict[str, Callable]

Raises**ValueError** – If no methods are registered for the type.**get_supported_operations()** → List[str]

Retrieve the list of supported operation names.

Returns

List of registered operation names.

Return type

List[str]

process_request (*request: Dict[str, Any]*) → Any

Process a request or sequence of requests.

Parameters

request (*Dict[str, Any]*) – The request dictionary specifying the operation, object, and attributes. For a single request, expected keys include “operation”, and optionally “obj”, “method” (str), “attributes” (dict). For a sequence of requests, expected format is {request_id: {sub_request}} where each sub_request has the same structure as a single request.

Returns

For a single request, a dictionary with status, object, method, result, and error (if status=False).

For a sequence of requests, a dictionary mapping request IDs to results.

Return type

Any

Raises

- **TypeError** – If the request is not a dictionary or contains invalid types.
- **ValueError** – If the request structure is invalid.

register_operation (*operation: str, super_instance: Callable*) → None

Register an operation with its super-instance handler.

Parameters

- **operation** (*str*) – The name of the operation.
- **super_instance** (*Callable*) – The super-instance with an ‘execute’ method.

Raises

ValueError – If the operation name is invalid or the super-instance lacks an ‘execute’ method.

set_managing_object (*obj: Any*) → None

Set the central managing object.

Parameters

obj (*Any*) – The object to set as the managing object.

update_registry (*additional_classes: List[Type] | None = None, clear_operations: bool = False*) → None

Update the method registry with additional base classes or clear operations.

Parameters

- **additional_classes** (*Optional[List[Type]]*) – Additional classes to register. Defaults to None.
- **clear_operations** (*bool*) – If True, clear all operations. Defaults to False.

common.super.project module

class `common.super.project.Project` (*name: str = 'DEFAULT_PROJECT', items: Dict[str, BaseEntity] | None = None*)

Bases: ABC

Abstract super-class for managing collections of BaseEntity items within a project using BaseContainer.

_name

The name of the project, must be a non-empty string.

Type

str

_items

Container of BaseEntity items indexed by their names.

Type

BaseContainer[*BaseEntity*]

_item_type

The type of items stored in the container, defaults to BaseEntity.

Type

Type[*BaseEntity*]

activate_all() → None

Activate all items in the container.

Raises

ValueError – If the container is empty.

add_item(item: BaseEntity) → None

Add a BaseEntity item to the project's container.

Parameters

item (*BaseEntity*) – The item to add.

Raises

- **TypeError** – If the item type does not match the expected type.
- **ValueError** – If an item with the same name already exists in the project.

clear() → None

Clear all items from container.

abstractmethod create_item(item_code: str = 'ITEM_DEFAULT', isactive: bool = True) → None

Create and add a new BaseEntity item to the project.

deactivate_all() → None

Deactivate all items in the container.

Raises

ValueError – If the container is empty.

drop_active() → None

Remove all active items from the container.

Raises

ValueError – If there are no active items.

drop_inactive() → None

Remove all inactive items from the container.

Raises

ValueError – If there are no inactive items.

abstractmethod classmethod from_dict(data: Dict[str, Any]) → *Project*

Create a project instance from a dictionary.

Parameters

data (*Dict*[*str*, *Any*]) – Dictionary with project configuration.

Returns

A new instance of the subclass initialized with the dictionary data.

Return type

Project

Raises

ValueError – If the data is invalid or cannot be deserialized.

`get_active_items()` → List[T]

Retrieve all active items in the container.

Returns

A list of items where isactive is True.

Return type

List[T]

`get_inactive_items()` → List[T]

Retrieve all inactive items in the container.

Returns

A list of items where isactive is False.

Return type

List[T]

`get_item(name: str)` → *BaseEntity*

Retrieve an item from the project by its name.

`get_items()` → Dict[str, *BaseEntity*]

Retrieve all items in the project as a dictionary.

`get_name()` → str

Retrieve the project's name.

`get_project()` → Dict[str, Any]

Get the entire project configuration as a dictionary.

`name: str`

`remove_item(name: str)` → None

Remove an item from the project by its name.

`set_item(name: str, item: BaseEntity)` → None

Set an item in the project by its name.

`set_name(name: str)` → None

Set the project's name.

`set_project(name: str, items: Dict[str, BaseEntity])` → None

Set the entire project configuration, replacing name and items.

`to_dict()` → Dict[str, Any]

Convert the project to a dictionary for serialization.

common.super.super module

class `common.super.super.Super` (*manipulator: Manipulator = None, methods: Dict[Type, Dict[str, Callable]] | None = None, cache_size: int = 2048*)

Bases: ABC

Abstract super-class providing common functionality for operation handlers.

Designed to work with a Manipulator, this class defines a framework for executing operations on objects based on attributes. Subclasses implement specific operations (e.g., configuration, inspection, calculation, etc.) by defining methods with naming conventions like `_<operation>_<type>` or `_<operation>`.

`_manipulator`

The associated Manipulator instance for method lookup.

Type

Manipulator

`_methods`

Custom method registry for specific object types.

Type

`Dict[Type, Dict[str, Callable]]`

`_operation`

The operation name, set by Manipulator during registration.

Type

`str`

Notes

- Method resolution order: explicit method, prefixed method (`_<operation>_<method>`), type-specific method (`_<operation>_<type>`), default method (`_<operation>`).
- Logging is integrated via `common.utils.logging_setup.logger`.
- Results are returned as dictionaries with keys: status (bool), object (str), method (str | None), result (Any), error (str | None, included only if status=False).

`execute` (*obj*: Any, *attributes*: Dict[str, Any] = None, *method*: str = None) → Dict[str, Any]

Execute an operation on an object based on attributes and an optional method.

Parameters

- **`obj`** (*Any*) – The object to process.
- **`attributes`** (*Dict[str, Any]*, *optional*) – Dictionary of operation attributes. Defaults to None.
- **`method`** (*str*, *optional*) – Explicit method to call, if provided in the request.

Returns

Dictionary with status, object (name), method, result, and error (if status=False).

Return type

`Dict[str, Any]`

`register_method` (*obj_type*: Type, *method_name*: str, *method*: Callable) → None

Register a custom method for a specific object type.

Parameters

- **`obj_type`** (*Type*) – The type of object the method applies to.
- **`method_name`** (*str*) – The name of the method.
- **`method`** (*Callable*) – The callable method to register.

2.1.2.2 Module contents

2.1.3 common.utils package

2.1.3.1 Submodules

common.utils.logging_setup module

`common.utils.logging_setup.setup_logging(log_file: str = 'output.log', log_level: int = 20, clear_log: bool = False) → Logger`

Set up and configure logging for the system.

Creates a logger with both file and console handlers, using a consistent format for log messages. Allows specifying the logging level and whether to clear the log file on start.

Parameters

- `log_file` (*str*) – Path to the log file. Defaults to “output.log”.
- `log_level` (*int*) – Logging level (e.g., logging.DEBUG, logging.INFO). Defaults to logging.INFO.
- `clear_log` (*bool*) – If True, clears the log file before adding new logs. Defaults to False.

Returns

The configured logger instance.

Return type

logging.Logger

Notes

- Log format: “%(asctime)s - %(name)s - %(levelname)s - %(message)s”.
- Handlers are added only if the logger has no existing handlers to avoid duplication.
- If `clear_log` is True, the log file is truncated before adding new logs.

`common.utils.logging_setup.update_logging_clear(log_file: str, clear_log: bool) → None`

Update the logging configuration to clear the log file if specified.

Parameters

- `log_file` (*str*) – Path to the log file.
- `clear_log` (*bool*) – If True, reconfigures the file handler to clear the log file.

`common.utils.logging_setup.update_logging_level(log_level: int) → None`

Update the logging level for the singleton logger and its handlers.

Parameters

- `log_level` (*int*) – New logging level (e.g., logging.DEBUG, logging.INFO).

Notes

- Updates the level of the singleton logger and all its handlers.
- If the logger is not initialized, it will be created with default settings and the specified level.

common.utils.validation module

`common.utils.validation.check_list_type(lst: list, expected_type, name: str) → None`

Check if all elements in a list or tuple match the expected type.

Parameters

- **lst** (*list*) – The list or tuple to check.
- **expected_type** – The expected type for all elements (e.g., str, int).
- **name** (*str*) – Name of the parameter for error messages.

Raises

TypeError – If *lst* is not a list/tuple or any element does not match *expected_type*.

Notes

- Logs an error message via *logger* before raising an exception.

Examples

```
>>> check_list_type(["a", "b"], str, "my_list")
>>> check_list_type([1, "b"], str, "my_list")
Traceback (most recent call last):
...
TypeError: All items in my_list must be of type <class 'str'>, got <class 'int'
↪ '>
```

`common.utils.validation.check_non_empty_string(value: str, name: str) → None`

Check if a value is a non-empty string.

Parameters

- **value** (*str*) – The value to check.
- **name** (*str*) – Name of the parameter for error messages.

Raises

- **TypeError** – If *value* is not a string.
- **ValueError** – If *value* is empty or contains only whitespace.

Notes

- Logs an error message via *logger* before raising an exception.

Examples

```
>>> check_non_empty_string("test", "my_string")
>>> check_non_empty_string("", "my_string")
Traceback (most recent call last):
...
ValueError: my_string must not be empty
```

`common.utils.validation.check_non_negative(value: float, name: str) → None`

Check if a numeric value is non-negative.

Parameters

- **value** (*float*) – The value to check.
- **name** (*str*) – Name of the parameter for error messages.

Raises

- **TypeError** – If value is not a number (int or float).
- **ValueError** – If value is negative (less than 0).

Notes

- Logs an error message via *logger* before raising an exception.

Examples

```
>>> check_non_negative(0.0, "my_value")
>>> check_non_negative(-1.0, "my_value")
Traceback (most recent call last):
...
ValueError: my_value must be non-negative, got -1.0
```

`common.utils.validation.check_non_zero(value: float, name: str) → None`

Check if a numeric value is non-zero.

Parameters

- **value** (*float*) – The value to check.
- **name** (*str*) – Name of the parameter for error messages.

Raises

- **TypeError** – If value is not a number (int or float).
- **ValueError** – If value is zero.

Notes

- Logs an error message via *logger* before raising an exception.

Examples

```
>>> check_non_zero(1.0, "my_value")
>>> check_non_zero(0.0, "my_value")
Traceback (most recent call last):
...
ValueError: my_value must be non-zero, got 0.0
```

`common.utils.validation.check_positive(value: float, name: str) → None`

Check if a numeric value is positive.

Parameters

- **value** (*float*) – The value to check.

- **name** (*str*) – Name of the parameter for error messages.

Raises

- **TypeError** – If value is not a number (int or float).
- **ValueError** – If value is not positive (less than or equal to 0).

Notes

- Logs an error message via *logger* before raising an exception.

Examples

```
>>> check_positive(1.5, "my_value")
>>> check_positive(0, "my_value")
Traceback (most recent call last):
...
ValueError: my_value must be positive, got 0
```

`common.utils.validation.check_range` (*value: float, min_val: float, max_val: float, name: str*) → `None`

Check if a numeric value is within a specified range.

Parameters

- **value** (*float*) – The value to check.
- **min_val** (*float*) – Minimum allowed value (inclusive).
- **max_val** (*float*) – Maximum allowed value (inclusive).
- **name** (*str*) – Name of the parameter for error messages.

Raises

- **TypeError** – If value is not a number (int or float).
- **ValueError** – If value is outside the range [*min_val*, *max_val*].

Notes

- Logs an error message via *logger* before raising an exception.

Examples

```
>>> check_range(5.0, 0.0, 10.0, "my_value")
>>> check_range(-1, 0.0, 10.0, "my_value")
Traceback (most recent call last):
...
ValueError: my_value must be between 0.0 and 10.0, got -1
```

`common.utils.validation.check_type` (*value, expected_type, name: str*) → `None`

Check if a value matches the expected type or is `None` for optional parameters.

Parameters

- **value** – The value to check.
- **expected_type** – The expected type or tuple of types (e.g., `str`, `(int, float)`).

- **name** (*str*) – Name of the parameter for use in error messages.

Raises

TypeError – If value is neither None nor of the expected type.

Notes

- Allows None as a valid value for optional parameters.
- Logs an error message via *logger* before raising an exception.

Examples

```
>>> check_type("test", str, "my_string")
>>> check_type(None, str, "optional_string") # No exception
>>> check_type(123, str, "my_string")
Traceback (most recent call last):
...
TypeError: my_string must be of type <class 'str'>, got <class 'int'>
```

2.1.3.2 Module contents**2.2 Module contents**

PYTHON MODULE INDEX

C

- `common`, 49
- `common.base`, 39
 - `common.base.basecontainer`, 31
 - `common.base.baseentity`, 36
- `common.super`, 45
 - `common.super.manipulator`, 39
 - `common.super.project`, 41
 - `common.super.super`, 43
- `common.utils`, 49
 - `common.utils.logging_setup`, 45
 - `common.utils.validation`, 46

p

- `pastrocore`, 30
 - `pastrocore.base`, 22
 - `pastrocore.base.frequencies`, 3
 - `pastrocore.base.observation`, 7
 - `pastrocore.base.scans`, 9
 - `pastrocore.base.sources`, 12
 - `pastrocore.base.spacetelescope`, 16
 - `pastrocore.base.telescope`, 17
 - `pastrocore.base.telescopes`, 20
 - `pastrocore.super`, 28
 - `pastrocore.super.schedule_calculator`, 22
 - `pastrocore.super.schedule_configurator`, 23
 - `pastrocore.super.schedule_inspector`, 23
 - `pastrocore.super.schedule_manipulator`, 24
 - `pastrocore.super.schedule_project`, 25
 - `pastrocore.super.schedule_visualizer`, 27
 - `pastrocore.utils`, 30
 - `pastrocore.utils.catalogmanager`, 28

Non-alphabetical

`_base_classes` (*common.super.manipulator.Manipulator attribute*), 39

`_cached_to_dict` (*common.base.basecontainer.BaseContainer attribute*), 31

`_fields` (*common.base.basecontainer.BaseContainer attribute*), 31

`_fields` (*common.base.baseentity.BaseEntity attribute*), 36

`_fields` (*common.base.baseentity.EntityMeta attribute*), 39

`_item_type` (*common.super.project.Project attribute*), 42

`_items` (*common.base.basecontainer.BaseContainer attribute*), 31

`_items` (*common.super.project.Project attribute*), 42

`_items` (*pastrocore.base.frequencies.Frequencies attribute*), 3

`_items` (*pastrocore.base.sources.Sources attribute*), 14

`_items` (*pastrocore.base.telescopes.Telescopes attribute*), 20

`_lock` (*pastrocore.super.schedule_calculator.ScheduleCalculator attribute*), 22

`_managing_object` (*common.super.manipulator.Manipulator attribute*), 39

`_manipulator` (*common.super.super.Super attribute*), 44

`_methods` (*common.super.super.Super attribute*), 44

`_name` (*common.super.project.Project attribute*), 41

`_operation` (*common.super.super.Super attribute*), 44

`_operations` (*common.super.manipulator.Manipulator attribute*), 39

`_registry` (*common.super.manipulator.Manipulator attribute*), 40

`_use_cache` (*common.base.basecontainer.BaseContainer attribute*), 31

A

`activate()` (*common.base.baseentity.BaseEntity method*), 37

`activate_all()` (*common.base.basecontainer.BaseContainer method*), 32

`activate_all()` (*common.super.project.Project method*), 42

`activate_all()` (*pastrocore.base.scans.Scans method*), 11

`activate_item()` (*common.base.basecontainer.BaseContainer method*), 32

`activate_item()` (*pastrocore.base.frequencies.Frequencies method*), 3

`activate_item()` (*pastrocore.base.sources.Sources method*), 15

`activate_item()` (*pastrocore.base.telescopes.Telescopes method*), 20

`add()` (*common.base.basecontainer.BaseContainer method*), 32

`add()` (*pastrocore.base.frequencies.Frequencies method*), 4

`add()` (*pastrocore.base.scans.Scans method*), 11

`add_flux()` (*pastrocore.base.sources.Source method*), 13

`add_item()` (*common.super.project.Project method*), 42

`add_item()` (*pastrocore.super.schedule_project.ScheduleProject method*), 25

`add_sefd()` (*pastrocore.base.telescope.Telescope method*), 18

`alt_name` (*pastrocore.base.sources.Source attribute*), 13

`azimuth_range` (*pastrocore.base.telescope.Telescope attribute*), 18

`AZIMUTHAL` (*pastrocore.base.spacetelescope.MountType attribute*), 16

`AZIMUTHAL` (*pastro-*

<i>core.base.telescope.MountType</i>	at-	<i>clear_catalogs()</i>	(<i>pastro-</i>
<i>tribute</i>), 17		<i>core.utils.catalogmanager.CatalogManager</i>	<i>method</i>), 28
B		<i>clear_flux_table()</i>	(<i>pastro-</i>
<i>bandwidth</i>	(<i>pastrocore.base.frequencies.IF</i>	<i>core.base.sources.Source</i>	<i>method</i>),
<i>attribute</i>), 6		13	
<i>base_classes</i>	(<i>pastro-</i>	<i>clear_sefd_table()</i>	(<i>pastro-</i>
	<i>core.super.schedule_manipulator.ScheduleManipulator</i>	<i>core.base.telescope.Telescope</i>	<i>method</i>),
	<i>attribute</i>), 24	18	
<i>BaseContainer</i>	(<i>class</i> <i>in</i> <i>com-</i>	<i>clone()</i>	(<i>common.base.basecontainer.BaseContainer</i>
	<i>mon.base.basecontainer</i>), 31	<i>clone()</i>	(<i>common.base.baseentity.BaseEntity</i>
<i>BaseEntity</i>	(<i>class</i> <i>in</i> <i>common.base.baseentity</i>),	<i>code</i>	(<i>pastrocore.base.observation.Observation</i>
36		<i>code</i>	<i>attribute</i>), 7, 8
C		<i>code</i>	(<i>pastrocore.base.telescope.Telescope</i>
<i>calculate_effective_area()</i>	(<i>pastro-</i>	<i>common</i>	<i>attribute</i>), 18
	<i>core.base.telescope.Telescope</i> <i>method</i>),	<i>module</i> , 49	
18		<i>common.base</i>	
<i>calculate_sefd()</i>	(<i>pastro-</i>	<i>module</i> , 39	
	<i>core.base.telescope.Telescope</i> <i>method</i>),	<i>common.base.basecontainer</i>	
18		<i>module</i> , 31	
<i>calculate_surface_efficiency()</i>	(<i>pastro-</i>	<i>common.base.baseentity</i>	
	<i>core.base.telescope.Telescope</i> <i>method</i>),	<i>module</i> , 36	
18		<i>common.super</i>	
<i>calculated_data</i>	(<i>pastro-</i>	<i>module</i> , 45	
	<i>core.base.observation.Observation</i>	<i>common.super.manipulator</i>	
<i>attribute</i>), 8		<i>module</i> , 39	
<i>CatalogManager</i>	(<i>class</i> <i>in</i> <i>pastro-</i>	<i>common.super.project</i>	
	<i>core.utils.catalogmanager</i>), 28	<i>module</i> , 41	
<i>check_activity_status()</i>	(<i>pastro-</i>	<i>common.super.super</i>	
	<i>core.base.scans.Scan</i> <i>method</i>), 9	<i>module</i> , 43	
<i>check_list_type()</i>	(<i>in</i> <i>module</i> <i>com-</i>	<i>common.utils</i>	
	<i>mon.utils.validation</i>), 46	<i>module</i> , 49	
<i>check_non_empty_string()</i>	(<i>in</i> <i>module</i> <i>com-</i>	<i>common.utils.logging_setup</i>	
	<i>mon.utils.validation</i>), 46	<i>module</i> , 45	
<i>check_non_negative()</i>	(<i>in</i> <i>module</i> <i>com-</i>	<i>common.utils.validation</i>	
	<i>mon.utils.validation</i>), 46	<i>module</i> , 46	
<i>check_non_zero()</i>	(<i>in</i> <i>module</i> <i>com-</i>	<i>copy()</i>	(<i>pastrocore.base.frequencies.Frequencies</i>
	<i>mon.utils.validation</i>), 47	<i>copy()</i>	<i>method</i>), 4
<i>check_positive()</i>	(<i>in</i> <i>module</i> <i>com-</i>	<i>copy()</i>	(<i>pastrocore.base.frequencies.IF</i> <i>method</i>),
	<i>mon.utils.validation</i>), 47	<i>copy()</i>	6
<i>check_range()</i>	(<i>in</i> <i>module</i> <i>com-</i>	<i>copy()</i>	(<i>pastrocore.base.observation.Observation</i>
	<i>mon.utils.validation</i>), 48	<i>copy()</i>	<i>method</i>), 8
<i>check_telescope_availability()</i>	(<i>pastro-</i>	<i>copy()</i>	(<i>pastrocore.base.scans.Scan</i> <i>method</i>), 10
	<i>core.base.scans.Scan</i> <i>method</i>), 10	<i>copy()</i>	(<i>pastrocore.base.scans.Scans</i> <i>method</i>),
<i>check_type()</i>	(<i>in</i> <i>module</i> <i>com-</i>	12	
	<i>mon.utils.validation</i>), 48	<i>copy()</i>	(<i>pastrocore.base.sources.Source</i>
<i>clear()</i>	(<i>common.base.basecontainer.BaseContainer</i>	<i>copy()</i>	<i>method</i>), 14
	<i>method</i>), 32	<i>copy()</i>	(<i>pastrocore.base.sources.Sources</i>
<i>clear()</i>	(<i>common.super.project.Project</i> <i>method</i>),	<i>copy()</i>	<i>method</i>), 15
42		<i>copy()</i>	(<i>pastrocore.base.telescopes.Telescopes</i>
<i>clear_calculated_data()</i>	(<i>pastro-</i>	<i>copy()</i>	<i>method</i>), 21
	<i>core.base.observation.Observation</i>	<i>copy()</i>	(<i>pastrocore.base.telescope.Telescope</i>
<i>method</i>), 8		<i>copy()</i>	<i>method</i>), 18
<i>clear_calculated_data()</i>	(<i>pastro-</i>	<i>create_if()</i>	(<i>pastro-</i>
	<i>core.super.schedule_project.ScheduleProject</i>	<i>core.base.frequencies.Frequencies</i>	
<i>method</i>), 25			

- method), 4
- create_item() (common.super.project.Project method), 42
- create_item() (pastrocore.super.schedule_project.ScheduleProject method), 25
- create_scan() (pastrocore.base.scans.Scans method), 12
- create_source() (pastrocore.base.sources.Sources method), 15
- create_telescope() (pastrocore.base.telescopes.Telescopes method), 21
- ## D
- de_d (pastrocore.base.sources.Source attribute), 13, 14
- de_m (pastrocore.base.sources.Source attribute), 13, 14
- de_s (pastrocore.base.sources.Source attribute), 13, 14
- deactivate() (common.base.baseentity.BaseEntity method), 37
- deactivate_all() (common.base.basecontainer.BaseContainer method), 33
- deactivate_all() (common.super.project.Project method), 42
- deactivate_all() (pastrocore.base.scans.Scans method), 12
- deactivate_item() (common.base.basecontainer.BaseContainer method), 33
- deactivate_item() (pastrocore.base.frequencies.Frequencies method), 4
- deactivate_item() (pastrocore.base.sources.Sources method), 15
- deactivate_item() (pastrocore.base.telescopes.Telescopes method), 21
- dec_degrees (pastrocore.base.sources.Source property), 14
- diameter (pastrocore.base.telescope.Telescope attribute), 18
- drop_active() (common.base.basecontainer.BaseContainer method), 33
- drop_active() (common.super.project.Project method), 42
- drop_inactive() (common.base.basecontainer.BaseContainer method), 33
- drop_inactive() (common.super.project.Project method), 42
- duration (pastrocore.base.scans.Scan attribute), 10
- ## E
- EARTH_DIAMETER (pastrocore.super.schedule_visualizer.ScheduleVisualizer attribute), 27
- effective_area_table (pastrocore.base.telescope.Telescope attribute), 18
- elevation_range (pastrocore.base.telescope.Telescope attribute), 18
- EntityMeta (class in common.base.baseentity), 39
- EQUATORIAL (pastrocore.base.spacetelescope.MountType attribute), 16
- EQUATORIAL (pastrocore.base.telescope.MountType attribute), 17
- execute() (common.super.super.Super method), 44
- execute() (pastrocore.super.schedule_visualizer.ScheduleVisualizer method), 27
- ## F
- flux_table (pastrocore.base.sources.Source attribute), 13, 14
- Frequencies (class in pastrocore.base.frequencies), 3
- frequencies (pastrocore.base.observation.Observation attribute), 7, 8
- frequencies (pastrocore.base.scans.Scan attribute), 10
- frequency (pastrocore.base.frequencies.IF attribute), 6
- from_dict() (common.base.basecontainer.BaseContainer class method), 33
- from_dict() (common.base.baseentity.BaseEntity class method), 37
- from_dict() (common.super.project.Project class method), 42
- from_dict() (pastrocore.base.observation.Observation class method), 8
- from_dict() (pastrocore.base.scans.Scan class method), 10
- from_dict() (pastrocore.base.scans.Scans class method), 12
- from_dict() (pastrocore.base.sources.Source class method), 14

from_dict() (astrocore.base.sources.Sources class method), 15	get_elevation_range() (astrocore.base.telescope.Telescope method), 19
from_dict() (astrocore.base.spacetelescope.SpaceTelescope class method), 16	get_end() (astrocore.base.scans.Scan method), 10
from_dict() (astrocore.base.telescopes.Telescopes class method), 22	get_flux() (astrocore.base.sources.Source method), 14
from_dict() (astrocore.base.telescope.Telescope class method), 18	get_frequencies() (astrocore.base.frequencies.Frequencies method), 4
from_dict() (astrocore.super.schedule_project.ScheduleProject class method), 25	get_frequencies() (astrocore.base.observation.Observation method), 9
	get_frequencies() (astrocore.base.scans.Scan method), 10
G	get_frequency_names() (astrocore.base.scans.Scan method), 10
get() (common.base.basecontainer.BaseContainer method), 33	get_frequency_wavelength() (astrocore.base.frequencies.IF method), 6
get() (common.base.baseentity.BaseEntity method), 37	get_inactive_items() (common.base.basecontainer.BaseContainer method), 34
get_active_items() (common.base.basecontainer.BaseContainer method), 34	get_inactive_items() (common.super.project.Project method), 43
get_active_items() (common.super.project.Project method), 43	get_inactive_scans() (astrocore.base.scans.Scans method), 12
get_active_scans() (astrocore.base.scans.Scans method), 12	get_item() (common.super.project.Project method), 43
get_all() (common.base.basecontainer.BaseContainer method), 34	get_items() (common.base.basecontainer.BaseContainer method), 34
get_azimuth_range() (astrocore.base.telescope.Telescope method), 18	get_items() (common.super.project.Project method), 43
get_bandwidths() (astrocore.base.frequencies.Frequencies method), 4	get_managing_object() (common.super.manipulator.Manipulator method), 40
get_by_value() (common.base.basecontainer.BaseContainer method), 34	get_methods_for_type() (common.super.manipulator.Manipulator method), 40
get_calculated_data() (astrocore.base.observation.Observation method), 8	get_MJD_endtime() (astrocore.base.scans.Scan method), 10
get_calculated_data_by_key() (astrocore.base.observation.Observation method), 8	get_MJD_starttime() (astrocore.base.scans.Scan method), 10
get_code() (astrocore.base.telescope.Telescope method), 18	get_name() (common.super.project.Project method), 43
get_coordinates() (astrocore.base.telescope.Telescope method), 19	get_observation() (astrocore.super.schedule_project.ScheduleProject method), 26
get_duration() (astrocore.base.observation.Observation method), 8	get_observation_by_code() (astrocore.super.schedule_project.ScheduleProject method), 26
get_duration() (astrocore.base.scans.Scan method), 10	get_observation_code() (astrocore.base.observation.Observation method), 9
get_effective_area() (astrocore.base.telescope.Telescope method),	get_observation_type() (astro-

<code>core.base.observation.Observation</code> method), 9	<code>get_telescope_names()</code> (pastro- <code>core.base.scans.Scan</code> method), 10
<code>get_orbit()</code> (pastro- <code>core.base.spacetelescope.SpaceTelescope</code> method), 16	<code>get_telescopes()</code> (pastro- <code>core.base.observation.Observation</code> method), 9
<code>get_pitch_range()</code> (pastro- <code>core.base.spacetelescope.SpaceTelescope</code> method), 17	<code>get_telescopes()</code> (pastrocore.base.scans.Scan method), 10
<code>get_polarizations()</code> (pastro- <code>core.base.frequencies.Frequencies</code> method), 5	<code>get_telescopes_by_type()</code> (pastro- <code>core.utils.catalogmanager.CatalogManager</code> method), 29
<code>get_project()</code> (<code>common.super.project.Project</code> method), 43	<code>get_velocities()</code> (pastro- <code>core.base.telescope.Telescope</code> method), 19
<code>get_project()</code> (pastro- <code>core.super.schedule_project.ScheduleProject</code> method), 26	<code>get_wavelengths()</code> (pastro- <code>core.base.frequencies.Frequencies</code> method), 5
<code>get_scans()</code> (pastro- <code>core.base.observation.Observation</code> method), 9	<code>get_yaw_range()</code> (pastro- <code>core.base.spacetelescope.SpaceTelescope</code> method), 17
<code>get_sefd()</code> (pastro- <code>core.base.telescope.Telescope</code> method), 19	H
<code>get_source()</code> (pastrocore.base.scans.Scan method), 10	<code>has_attribute()</code> (<code>com- mon.base.baseentity.BaseEntity</code> method), 38
<code>get_source()</code> (pastro- <code>core.utils.catalogmanager.CatalogManager</code> method), 28	<code>has_item()</code> (<code>common.base.basecontainer.BaseContainer</code> method), 34
<code>get_source_name()</code> (pastro- <code>core.base.scans.Scan</code> method), 10	I
<code>get_sources()</code> (pastro- <code>core.base.observation.Observation</code> method), 9	<code>IF</code> (<code>class in pastrocore.base.frequencies</code>), 5
<code>get_sources_by_dec_range()</code> (pastro- <code>core.utils.catalogmanager.CatalogManager</code> method), 29	<code>interpolation_method</code> (pastro- <code>core.base.spacetelescope.SpaceTelescope</code> attribute), 17
<code>get_sources_by_ra_range()</code> (pastro- <code>core.utils.catalogmanager.CatalogManager</code> method), 29	<code>is_off_source</code> (pastrocore.base.scans.Scan at- tribute), 10
<code>get_start()</code> (pastrocore.base.scans.Scan method), 10	<code>isactive</code> (<code>common.base.baseentity.BaseEntity</code> attribute), 36, 38
<code>get_start_datetime()</code> (pastro- <code>core.base.observation.Observation</code> method), 9	<code>isactive</code> (pastrocore.base.frequencies.Frequencies attribute), 3, 5
<code>get_style_config()</code> (pastro- <code>core.super.schedule_visualizer.ScheduleVisualizer</code> method), 27	<code>isactive</code> (pastrocore.base.frequencies.IF at- tribute), 6
<code>get_supported_operations()</code> (<code>com- mon.super.manipulator.Manipulator</code> method), 40	<code>isactive</code> (pastrocore.base.observation.Observation attribute), 8
<code>get_surface_efficiency()</code> (pastro- <code>core.base.telescope.Telescope</code> method), 19	<code>isactive</code> (pastrocore.base.scans.Scans at- tribute), 12
<code>get_system_temperature()</code> (pastro- <code>core.base.telescope.Telescope</code> method), 19	<code>isactive</code> (pastrocore.base.sources.Source attribute), 13
<code>get_telescope()</code> (pastro- <code>core.utils.catalogmanager.CatalogManager</code> method), 29	<code>isactive</code> (pastrocore.base.sources.Sources at- tribute), 14, 15
	<code>isactive</code> (pastrocore.base.telescopes.Telescopes attribute), 20, 22
	<code>items</code> (<code>common.base.basecontainer.BaseContainer</code> property), 35
	K
	<code>kepler_elements</code> (pastro- <code>core.base.spacetelescope.SpaceTelescope</code> attribute), 17

L

`load_source_catalog()` (*pastrocore.utils.catalogmanager.CatalogManager* method), 29

`load_telescope_catalog()` (*pastrocore.utils.catalogmanager.CatalogManager* method), 30

M

`managing_object` (*pastrocore.super.schedule_manipulator.ScheduleManipulator* attribute), 24

`Manipulator` (class in *common.super.manipulator*), 39

`manipulator` (*pastrocore.super.schedule_calculator.ScheduleCalculator* attribute), 22

`module`

`common`, 49

`common.base`, 39

`common.base.basecontainer`, 31

`common.base.baseentity`, 36

`common.super`, 45

`common.super.manipulator`, 39

`common.super.project`, 41

`common.super.super`, 43

`common.utils`, 49

`common.utils.logging_setup`, 45

`common.utils.validation`, 46

`pastrocore`, 30

`pastrocore.base`, 22

`pastrocore.base.frequencies`, 3

`pastrocore.base.observation`, 7

`pastrocore.base.scans`, 9

`pastrocore.base.sources`, 12

`pastrocore.base.spacetelescope`, 16

`pastrocore.base.telescope`, 17

`pastrocore.base.telescopes`, 20

`pastrocore.super`, 28

`pastrocore.super.schedule_calculator`, 22

`pastrocore.super.schedule_configurator`, 23

`pastrocore.super.schedule_inspector`, 23

`pastrocore.super.schedule_manipulator`, 24

`pastrocore.super.schedule_project`, 25

`pastrocore.super.schedule_visualizer`, 27

`pastrocore.utils`, 30

`pastrocore.utils.catalogmanager`, 28

`mount_type` (*pastrocore.base.telescope.Telescope* attribute), 19

`MountType` (class in *pastrocore.base.spacetelescope*), 16

`MountType` (class in *pastrocore.base.telescope*), 17

`name` (*common.base.basecontainer.BaseContainer* attribute), 35

`name` (*common.base.baseentity.BaseEntity* attribute), 36, 38

`name` (*common.super.project.Project* attribute), 43

`name` (*pastrocore.base.frequencies.Frequencies* attribute), 3, 5

`name` (*pastrocore.base.frequencies.IF* attribute), 6, 7

`name` (*pastrocore.base.observation.Observation* attribute), 9

`name` (*pastrocore.base.scans.Scan* attribute), 10

`name` (*pastrocore.base.scans.Scans* attribute), 12

`name` (*pastrocore.base.sources.Source* attribute), 12, 14

`name` (*pastrocore.base.sources.Sources* attribute), 15

`name` (*pastrocore.base.telescopes.Telescopes* attribute), 22

`name` (*pastrocore.base.telescope.Telescope* attribute), 19

`name` (*pastrocore.super.schedule_project.ScheduleProject* attribute), 26

`name_J2000` (*pastrocore.base.sources.Source* attribute), 13, 14

O

`Observation` (class in *pastrocore.base.observation*), 7

`observation_type` (*pastrocore.base.observation.Observation* attribute), 7, 9

`operations` (*pastrocore.super.schedule_manipulator.ScheduleManipulator* attribute), 24

`orbit_file` (*pastrocore.base.spacetelescope.SpaceTelescope* attribute), 17

P

`pastrocore` module, 30

`pastrocore.base` module, 22

`pastrocore.base.frequencies` module, 3

`pastrocore.base.observation` module, 7

`pastrocore.base.scans` module, 9

`pastrocore.base.sources` module, 12

`pastrocore.base.spacetelescope` module, 16

pastrocore.base.telescope
 module, 17
 pastrocore.base.telescopes
 module, 20
 pastrocore.super
 module, 28
 pastrocore.super.schedule_calculator
 module, 22
 pastrocore.super.schedule_configurator
 module, 23
 pastrocore.super.schedule_inspector
 module, 23
 pastrocore.super.schedule_manipulator
 module, 24
 pastrocore.super.schedule_project
 module, 25
 pastrocore.super.schedule_visualizer
 module, 27
 pastrocore.utils
 module, 30
 pastrocore.utils.catalogmanager
 module, 28
 pitch_range (pastrocore.base.spacetelescope.SpaceTelescope attribute), 17
 polarizations (pastrocore.base.frequencies.IF attribute), 6, 7
 process_request() (common.super.manipulator.Manipulator method), 40
 Project (class in common.super.project), 41

R

ra_degrees (pastrocore.base.sources.Source property), 14
 ra_h (pastrocore.base.sources.Source attribute), 12, 14
 ra_m (pastrocore.base.sources.Source attribute), 12, 14
 ra_s (pastrocore.base.sources.Source attribute), 13, 14
 register_method() (common.super.super.Super method), 44
 register_operation() (common.super.manipulator.Manipulator method), 41
 remove() (common.base.basecontainer.BaseContainer method), 35
 remove() (pastrocore.base.scans.Scans method), 12
 remove_flux() (pastrocore.base.sources.Source method), 14
 remove_item() (common.super.project.Project method), 43
 remove_sefd() (pastrocore.base.telescope.Telescope method), 19
 Scan (class in pastrocore.base.scans), 9
 Scans (class in pastrocore.base.scans), 11
 scans (pastrocore.base.observation.Observation attribute), 8, 9
 ScheduleCalculator (class in pastrocore.super.schedule_calculator), 22
 ScheduleConfigurator (class in pastrocore.super.schedule_configurator), 23
 ScheduleInspector (class in pastrocore.super.schedule_inspector), 23
 ScheduleManipulator (class in pastrocore.super.schedule_manipulator), 24
 ScheduleProject (class in pastrocore.super.schedule_project), 25
 ScheduleVisualizer (class in pastrocore.super.schedule_visualizer), 27
 sefd_table (pastrocore.base.telescope.Telescope attribute), 19
 set() (common.base.basecontainer.BaseContainer method), 35
 set() (common.base.baseentity.BaseEntity method), 38
 set() (pastrocore.base.telescope.Telescope method), 19
 set_calculated_data_by_key() (pastrocore.base.observation.Observation method), 9
 set_dec_degrees() (pastrocore.base.sources.Source method), 14
 set_duration() (pastrocore.base.scans.Scan method), 10
 set_frequencies() (pastrocore.base.scans.Scan method), 10
 set_frequency_wavelength() (pastrocore.base.frequencies.IF method), 7
 set_if() (pastrocore.base.frequencies.Frequencies method), 5
 set_interpolation_method() (pastrocore.base.spacetelescope.SpaceTelescope method), 17
 set_item() (common.base.basecontainer.BaseContainer method), 35
 set_item() (common.super.project.Project method), 43
 set_item() (pastrocore.super.schedule_project.ScheduleProject method), 26
 set_items() (common.base.basecontainer.BaseContainer method), 35
 set_items() (pastrocore.base.frequencies.Frequencies

method), 5

set_keplerian() (pastrocore.base.spacetelescope.SpaceTelescope method), 17

set_managing_object() (common.super.manipulator.Manipulator method), 41

set_name() (common.super.project.Project method), 43

set_orbit() (pastrocore.base.spacetelescope.SpaceTelescope method), 17

set_project() (common.super.project.Project method), 43

set_project() (pastrocore.super.schedule_project.ScheduleProject method), 26

set_ra_degrees() (pastrocore.base.sources.Source method), 14

set_scan() (pastrocore.base.scans.Scans method), 12

set_source() (pastrocore.base.scans.Scan method), 10

set_source() (pastrocore.base.sources.Sources method), 15

set_start() (pastrocore.base.scans.Scan method), 11

set_style_config() (pastrocore.super.schedule_visualizer.ScheduleVisualizer method), 27

set_telescopes() (pastrocore.base.scans.Scan method), 11

setup_logging() (in module common.utils.logging_setup), 45

Source (class in pastrocore.base.sources), 12

source (pastrocore.base.scans.Scan attribute), 11

source_catalog (pastrocore.utils.catalogmanager.CatalogManager attribute), 28

Sources (class in pastrocore.base.sources), 14

sources (pastrocore.base.observation.Observation attribute), 7, 9

SPACE (pastrocore.base.spacetelescope.MountType attribute), 16

SPACE (pastrocore.base.telescope.MountType attribute), 17

SpaceTelescope (class in pastrocore.base.spacetelescope), 16

spectral_index (pastrocore.base.sources.Source attribute), 13, 14

SPEED_OF_LIGHT (pastrocore.super.schedule_visualizer.ScheduleVisualizer attribute), 27

start (pastrocore.base.scans.Scan attribute), 11

Super (class in common.super.super), 43

surface_accuracy (pastrocore.base.telescope.Telescope attribute), 19

surface_efficiency_table (pastrocore.base.telescope.Telescope attribute), 19

sync_with_observation() (pastrocore.base.scans.Scan method), 11

system_temperature_table (pastrocore.base.telescope.Telescope attribute), 19

T

Telescope (class in pastrocore.base.telescope), 17

telescope_catalog (pastrocore.utils.catalogmanager.CatalogManager attribute), 28

Telescopes (class in pastrocore.base.telescopes), 20

telescopes (pastrocore.base.observation.Observation attribute), 7, 9

telescopes (pastrocore.base.scans.Scan attribute), 11

time_execution() (in module pastrocore.super.schedule_calculator), 22

to_dict() (common.base.basecontainer.BaseContainer method), 35

to_dict() (common.base.baseentity.BaseEntity method), 38

to_dict() (common.super.project.Project method), 43

to_dict() (pastrocore.base.observation.Observation method), 9

to_dict() (pastrocore.base.scans.Scan method), 11

to_dict() (pastrocore.base.spacetelescope.SpaceTelescope method), 17

to_dict() (pastrocore.base.telescope.Telescope method), 19

to_dict() (pastrocore.super.schedule_project.ScheduleProject method), 27

type (pastrocore.base.spacetelescope.SpaceTelescope attribute), 17

type (pastrocore.base.telescope.Telescope attribute), 19

U

update_logging_clear() (in module common.utils.logging_setup), 45

update_logging_level() (in module common.utils.logging_setup), 45

update_registry() (common.super.manipulator.Manipulator method), 41

`use_kep` (*pastrocore.base.spacetelescope.SpaceTelescope attribute*), [17](#)

V

`validate()` (*pastrocore.base.observation.Observation method*), [9](#)

`validate_with_observation()` (*pastrocore.base.scans.Scan method*), [11](#)

`vx` (*pastrocore.base.telescope.Telescope attribute*), [19](#)

`vy` (*pastrocore.base.telescope.Telescope attribute*), [20](#)

`vz` (*pastrocore.base.telescope.Telescope attribute*), [20](#)

X

`x` (*pastrocore.base.telescope.Telescope attribute*), [20](#)

Y

`y` (*pastrocore.base.telescope.Telescope attribute*), [20](#)

`yaw_range` (*pastrocore.base.spacetelescope.SpaceTelescope attribute*), [17](#)

Z

`z` (*pastrocore.base.telescope.Telescope attribute*), [20](#)